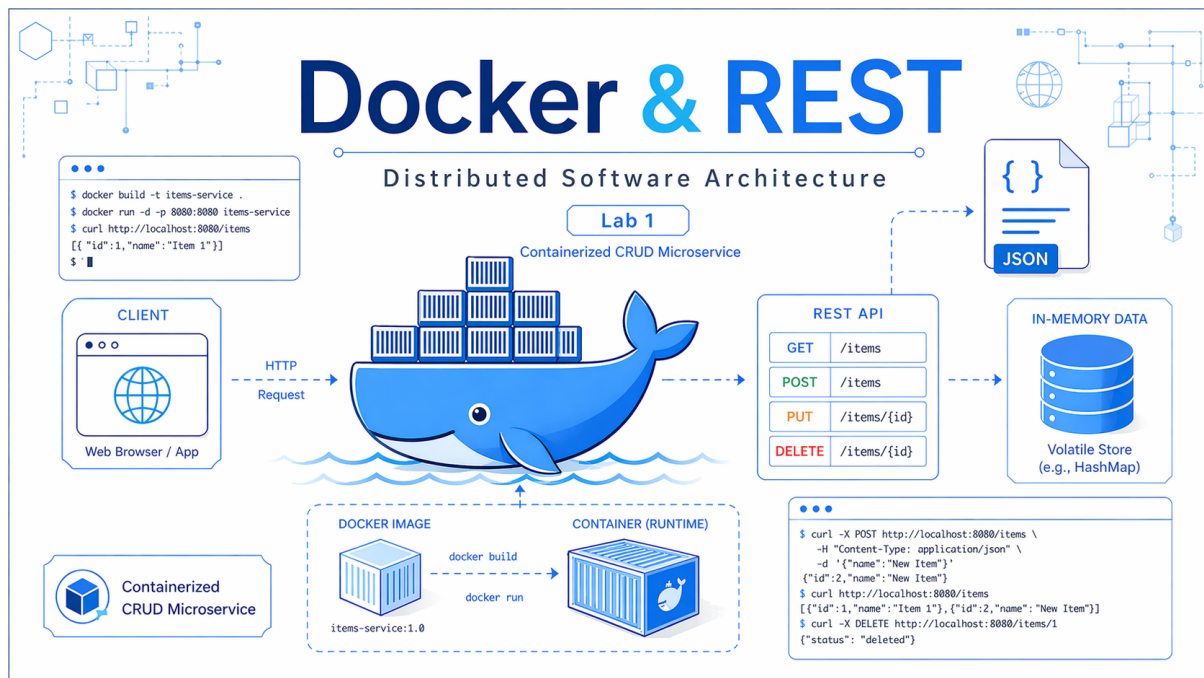


Distributed Software Architecture

Lab 1

Docker & REST

Prof. Dr. Dmitry Kachan



Docker & REST

Overview and Goals

In this first lab, you create the first small service of your semester project. The service itself is intentionally simple. The main goal is not advanced programming, but understanding how a microservice is exposed through HTTP, packaged as a container image, started as a container, tested from outside, and inspected through logs and Docker networking.

- Choose a small project story that can continue across the following labs.
- Implement one simple REST microservice with basic CRUD operations.
- Containerize this service with a Dockerfile.
- Run and test the service locally and with Docker.
- Explain the path of a request from the client to the containerized service.

Important note: The result of this lab will be reused in later labs. Store your code, Dockerfile, documentation, and test evidence in a Git repository <https://gitlab.hs-anhalt.de> or, if you prefer, in a public repository such as <https://github.com>. If Git is new to you, use Appendix 1: Useful Git Commands.

What This Lab Is Really About

This lab is not a programming exam. You are allowed to use the provided Flask skeleton as a starting point. Your own work is required in the following parts:

- adapting the example to your own project story;
- completing the missing CRUD endpoints;
- defining meaningful validation rules for your domain object;
- building and running the service as a Docker container;
- explaining what happens when a request reaches the container;
- documenting your tests and observations.

Focus for This Lab

Concentrate on the **core learning path**:

1. choose a small project story,
2. implement the REST API,
3. test it locally,
4. containerize it with Docker,
5. run and test the container,
6. explain the request path.

Support material is moved to clickable appendix references, for example Git commands, Dockerfile and dockerignore notes, Docker commands, and useful tools.

Recommended Version Baseline

The examples in this lab use the following baseline:

- Docker Engine or Docker Desktop with the `docker` command available in the terminal.
- Python image `python:3.13.13-slim` as the container base image.
- Flask 3.1.3 as the REST framework.
- Waitress 3.0.2 as the WSGI server used inside the container.

Local Python 3.12 or newer is acceptable for development. The container is the reference runtime for grading. If the tools are unfamiliar, see Appendix 1: Useful Tools.

Lab Description

1 Choose a Semester Project Story

The course labs should become one coherent application, not several unrelated exercises. In later labs, this first service will be extended with internal service-to-service communication, persistence, reliability mechanisms, observability, event-driven communication, security, and multi-container orchestration.

1. **Choose one project story.** You may use one of these proposals or define your own comparable idea:

Pet shelter / adoption platform: manage pets, adoption status, appointments, and later notifications.

Campus task quest: manage study tasks or missions, progress states, priorities, and later reminders.

Smart greenhouse: manage plants, sensors, watering zones, and later sensor events or control decisions.

Mini library or game inventory: manage books, board games, loans, reservations, and later user roles.

Event planner: manage events, registrations, capacity, and later asynchronous notifications.

2. **Define your first domain object.** For easier grading, the HTTP path may still be called `/items`. However, the object content must match your story. Your object must contain at least:

- `id` – generated by the service;
- `name` – a human-readable name;
- `status` – a domain-specific state, for example `available`, `reserved`, `open`, `done`, `healthy`, or `warning`;
- one additional domain-specific field, for example `species`, `priority`, `location`, `category`, or `date`.

3. **Create a short story document** in `docs/story.md`. It must contain:

- project title;
- selected domain object;
- two or three users or actors;
- three short use cases;
- one sentence explaining how the system could later benefit from another microservice, events, security, and monitoring.

Example Domain Objects

```

1 # Pet shelter example
2 {
3     "id": 1,
4     "name": "Bella",
5     "status": "available",
6     "species": "dog"
7 }
8
9 # Smart greenhouse example
10 {
11     "id": 1,
12     "name": "Tomato Bed A",
13     "status": "healthy",
14     "location": "greenhouse-zone-1"
15 }

```

⚠ Keep the Scope Small

Do not build a frontend, a database, login, user management, or complex business logic in this lab. The first service should be small enough that you can focus on REST, containers, and request flow.

2 Prepare the Project Structure

Support: for repository commands, see Appendix 1: Useful Git Commands.

1. **Create a repository or project directory.** A structure that can grow in later labs is recommended:

Recommended Project Structure

```

1 dsa-semester-project/
2   catalog-service/
3     app.py
4     requirements.txt
5     Dockerfile
6   docs/
7     story.md
8   README.md

```

2. **Use Python with Flask unless you have a strong reason to use another stack.** Python + Flask keeps the code short, so the microservice and Docker concepts remain visible. You may use Node.js, Go, Java, or another stack, but the HTTP behavior and container behavior must be equivalent. For Flask route basics, see Appendix 1: Flask Basics.
3. **Create the dependency file** inside `catalog-service/`:

requirements.txt

```

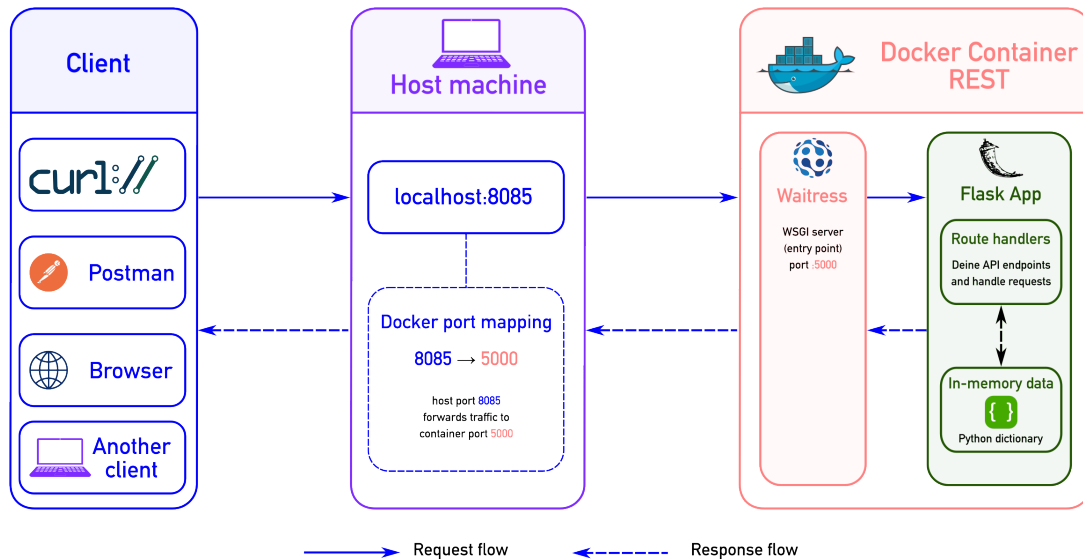
1 Flask==3.1.3
2 waitress==3.0.2

```

3 Target Lab Architecture

The goal of this lab is a **single small containerized REST service**. The picture for this section should show the architecture that you are expected to achieve by the end of the lab.

The important idea is the complete path of one request: a client sends an HTTP request to `localhost:8085`, Docker forwards the request from host port 8085 to container port 5000, Waitress serves the Flask



application, the application accesses an in-memory data structure, and a JSON response is returned to the client. This picture should help students understand the runtime architecture, not just the source code. For the port-mapping idea, see Appendix 1: Host Port vs. Container Port.

4 Understand the Request Path

Before writing much code, understand what you are building:

Request Path in This Lab

```

1  curl / Postman / Browser
2      |
3      v
4  http://localhost:8085
5      |
6      v
7  Docker port mapping: host 8085 -> container 5000
8      |
9      v
10 Waitress process inside the container, listening on port 5000
11     |
12     v
13 Flask application object: app
14     |
15     v
16 Route function, in-memory dictionary, JSON response
  
```

This request path is more important than the example object itself. Later labs will add more services and more communication paths. If the difference between 8085 and 5000 is unclear, see Appendix 1: Host Port vs. Container Port. If Waitress is unclear, see Appendix 1: Why Waitress Is Used.

5 Implement a Basic REST Service

Support: for Flask route decorators and JSON responses, see Appendix 1: Flask Basics. For HTTP methods and cURL, see Appendix 1: HTTP and cURL Mini-Cheat Sheet.

1. **Implement an in-memory API.** Your service manages the main domain object from your story. Store objects in a Python dictionary or a similar in-memory structure. The data may disappear when the service restarts. This is expected in Lab 1.

2. Required endpoints:

- GET /health – health check for the service.
- GET /items – list all objects.
- GET /items/{id} – retrieve one object by ID.
- POST /items – create a new object; the service generates the ID.
- PUT /items/{id} – update an existing object; the ID must not change.
- DELETE /items/{id} – delete an existing object.

3. Required status codes:

- 200 OK for successful read and update operations.
- 201 Created for successful creation with POST /items.
- 204 No Content for successful deletion with DELETE /items/{id}.
- 400 Bad Request for malformed JSON or missing/invalid required fields.
- 404 Not Found if the requested object does not exist.
- 415 Unsupported Media Type if a request body is sent without Content-Type: application/json.

4. Use a consistent error format. For example:**Example Error Response**

```

1 {
2   "error": {
3     "code": "validation_error",
4     "message": "Field 'name' is required."
5   }
6 }
```

6 Beginner-Friendly Implementation Path

If you have little programming experience, do not try to implement everything at once. Use this order. If a Flask term in the skeleton is unclear, open Appendix 1: Flask Basics.

1. Start with GET /health.
2. Add GET /items with an empty dictionary.
3. Add POST /items and create one object.
4. Add GET /items/{id} by copying the lookup pattern.
5. Add PUT /items/{id} by combining lookup and validation.
6. Add DELETE /items/{id} only after the previous steps work.
7. Only then build the Docker image.

Flask Starter Skeleton – Incomplete on Purpose

```

1 import logging
2 import os
3
4 from flask import Flask, jsonify, request
5
```

```

6 logging.basicConfig(level=logging.INFO)
7
8 app = Flask(__name__)
9
10 SERVICE_NAME = os.getenv("SERVICE_NAME", "catalog-service")
11
12 items = {}
13 next_id = 1
14
15
16 def error_response(code, message, status):
17     return jsonify({"error": {"code": code, "message": message}}), status
18
19
20 def read_json_body():
21     if not request.is_json:
22         return None, error_response(
23             "unsupported_media_type",
24             "Content-Type must be application/json.",
25             415,
26         )
27
28     data = request.get_json(silent=True)
29     if not isinstance(data, dict):
30         return None, error_response(
31             "invalid_json",
32             "Request body must be a valid JSON object.",
33             400,
34         )
35
36     return data, None
37
38
39 def validate_item_input(data):
40     name = data.get("name")
41     if not isinstance(name, str) or not name.strip():
42         return "Field 'name' is required and must be a non-empty string."
43
44     status = data.get("status")
45     if not isinstance(status, str) or not status.strip():
46         return "Field 'status' is required and must be a non-empty string."
47
48     # TODO: add one validation rule for your domain-specific field.
49     # Example for a pet shelter:
50     # species = data.get("species")
51     # if not isinstance(species, str) or not species.strip():
52     #     return "Field 'species' is required."
53
54     return None
55
56
57 @app.get("/health")
58 def health():
59     return jsonify({"status": "ok", "service": SERVICE_NAME}), 200
60
61
62 @app.get("/items")
63 def list_items():
64     return jsonify(list(items.values())), 200
65
66
67 @app.post("/items")
68 def create_item():

```

```

69     global next_id
70
71     data, problem = read_json_body()
72     if problem is not None:
73         return problem
74
75     validation_error = validate_item_input(data)
76     if validation_error is not None:
77         return error_response("validation_error", validation_error, 400)
78
79     item = {
80         "id": next_id,
81         "name": data["name"].strip(),
82         "status": data["status"].strip(),
83         # TODO: add your domain-specific field here.
84         # Example: "species": data["species"].strip(),
85     }
86
87     items[next_id] = item
88     next_id += 1
89
90     app.logger.info("Created item with id=%s", item["id"])
91     return jsonify(item), 201
92
93
94 @app.get("/items/<int:item_id>")
95 def get_item(item_id):
96     # TODO: return the item if it exists; otherwise return 404.
97     pass
98
99
100 @app.put("/items/<int:item_id>")
101 def update_item(item_id):
102     # TODO: check that the item exists.
103     # TODO: read and validate the JSON body.
104     # TODO: update the item but keep the same id.
105     pass
106
107
108 @app.delete("/items/<int:item_id>")
109 def delete_item(item_id):
110     # TODO: delete the item if it exists; otherwise return 404.
111     # A successful DELETE must return an empty body and status code 204.
112     pass
113
114
115 if __name__ == "__main__":
116     app.run(host="0.0.0.0", port=5000, debug=True)

```

⚠ Why the Skeleton Is Not a Full Solution

The skeleton gives you the shape of a small REST service. You must still complete the missing methods, add your own domain field, add validation for that field, and test success and error cases. This keeps the programming part manageable while still requiring your own engineering work. For the lookup pattern, see Appendix 1: Flask Basics.

7 Test Locally Before Containerization

This step is recommended because it separates application errors from Docker errors. During local testing, the Flask development server runs directly on your machine at `localhost:5000`. After containeriza-

tion, the same service will be reached through Docker at `localhost:8085`. For HTTP tests, see Appendix 1: HTTP and cURL Mini-Cheat Sheet.

1. Start the service locally from inside `catalog-service/`:

Run Locally with Flask Development Server

```
1 python app.py
```

The local command uses Flask's development server. The Docker container later uses Waitress instead.

2. Test the health endpoint:

Health Check

```
1 curl -i http://localhost:5000/health
```

3. Create a domain object. Adapt the JSON fields to your story:

Create Object

```
1 curl -i -X POST http://localhost:5000/items \
2   -H "Content-Type: application/json" \
3   -d '{"name":"Bella","status":"available","species":"dog"}'
```

4. List all objects:

List Objects

```
1 curl -i http://localhost:5000/items
```

5. Trigger one error intentionally:

Intentional Error Case

```
1 curl -i -X POST http://localhost:5000/items \
2   -H "Content-Type: application/json" \
3   -d '{"status":"available"}'
```

8 Containerize and Run with Docker

Support: For Dockerfile instructions and `.dockerignore`, see Appendix 1: Dockerfile and `dockerignore` Extras. For environment variables, see Appendix 1: Environment Variables. For Docker CLI commands, see Appendix 1: Useful Docker Commands.

1. Create a Dockerfile inside `catalog-service/`.

Dockerfile

```
1 FROM python:3.13.13-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1
4 ENV PYTHONUNBUFFERED=1
5 ENV SERVICE_NAME=catalog-service
6 ENV PORT=5000
7
8 WORKDIR /app
9
10 COPY requirements.txt .
11 RUN python -m pip install --no-cache-dir -r requirements.txt
12
13 RUN useradd --create-home --shell /usr/sbin/nologin appuser
14
```

```

15 COPY --chown=appuser:appuser . .
16
17 USER appuser
18
19 EXPOSE 5000
20
21 CMD ["sh", "-c", "exec waitress-serve --host=0.0.0.0 --port=${PORT:-5000} app:app"]

```

2. **Build the Docker image** from inside the `catalog-service/` directory:

Build Image

```
1 docker build --pull -t dsa-lab1-catalog:1.0 .
```

3. **Run the container** in detached mode. The mapping `-p 8085:5000` means that the host listens on port 8085 and forwards traffic into container port 5000; see Appendix 1: Host Port vs. Container Port.

Run Container

```

1 docker rm -f dsa-lab1-catalog 2>/dev/null || true
2
3 docker run -d \
4   --name dsa-lab1-catalog \
5   -p 8085:5000 \
6   -e SERVICE_NAME=catalog-service \
7   dsa-lab1-catalog:1.0

```

4. **Verify functionality inside the container.** Now use `localhost:8085`, not `localhost:5000`, because the request enters through the host port; see Appendix 1: Host Port vs. Container Port.

Containerized API Test

```

1 curl -i http://localhost:8085/health
2 curl -i http://localhost:8085/items
3
4 curl -i -X POST http://localhost:8085/items \
5   -H "Content-Type: application/json" \
6   -d '{"name":"Bella","status":"available","species":"dog"}'

```

5. **Check the logs.** If the command is unclear, see Appendix 1: Useful Docker Commands.

Read Container Logs

```
1 docker logs dsa-lab1-catalog
```

Store Your Results

Keep the repository, story document, source code, Dockerfile, and test evidence. Future labs will extend this service instead of starting again from zero.

Deliverables to Submit

- **Story document:** `docs/story.md` with selected domain, actors, use cases, and future extension idea.
- **Source code:** REST service implementing all required endpoints.
- **Dependency file:** `requirements.txt` or equivalent for another language.
- **Dockerfile:** container build configuration. Optional but recommended: add `.dockerignore`; see Appendix 1: Dockerfile and `dockerignore` Extras.

- **Testing evidence:**
 - one successful `POST` request;
 - one successful `GET /items` request;
 - one successful `GET /items/{id}` request;
 - one successful `PUT` or `DELETE` request;
 - one error case, for example missing `name` or wrong `Content-Type`;
 - container logs or console output showing the service running in Docker.
- **Short explanation:** 5–10 sentences explaining the request path from client to containerized Flask application, and why port mapping `-p 8085:5000` is required. Use Appendix 1: Host Port vs. Container Port only as support; write the explanation in your own words.

Lab Timing Guidance (3 Hours)

- **0–20 min:** Choose project story and define the domain object.
- **20–40 min:** Set up repository structure and dependency files.
- **40–100 min:** Implement the REST endpoints using the beginner-friendly path.
- **100–125 min:** Test locally with `curl`, `HTTPIe`, `Postman`, or `Insomnia`.
- **125–160 min:** Write `Dockerfile`, build image, run container, and verify logs.
- **160–180 min:** Collect test evidence and write the short explanation.

Appendix 1: Optional Support Material

This appendix is optional. It is provided for students who want extra support or want to go a little further.

1 Flask Basics for This Lab

- `app = Flask(__name__)` creates the Flask application object. Waitress later imports this object as `app:app`.
- `@app.get("/items")` registers a function for an HTTP GET request on `/items`.
- `@app.post("/items")` registers a function for an HTTP POST request on `/items`.
- `request.is_json` checks whether the request declares a JSON body.
- `request.get_json(silent=True)` tries to parse the JSON body and returns `None` instead of raising an exception when parsing fails.
- `jsonify(...)` creates a JSON HTTP response.
- Returning `(body, status_code)` from a route allows you to control the HTTP status code.
- The in-memory dictionary resets when the process or container restarts. This is expected in Lab 1; persistence will be addressed later.

Minimal Lookup Pattern

```

1 @app.get("/items/<int:item_id>")
2 def get_item(item_id):
3     item = items.get(item_id)
4     if item is None:
5         return error_response(
6             "not_found",
7             f"Item with id {item_id} does not exist.",
8             404,
9         )
10    return jsonify(item), 200

```

2 Why Waitress Is Used

Waitress is not imported in `app.py`. It is used as the process that starts the web application inside the container:

Waitress in the Dockerfile CMD

```

1 CMD ["sh", "-c", "exec waitress-serve --host=0.0.0.0 --port=${PORT:-5000} app:app"]

```

The final part, `app:app`, means:

- load the Python file `app.py`;
- inside that file, find the Flask application object named `app`;
- serve that Flask application on all container network interfaces, port 5000 by default.

3 Host Port vs. Container Port

Docker containers have their own network namespace. A service can listen on one port *inside* the container, while Docker publishes it on another port on the host.

- 5000 is the **container port**: Waitress listens on this port inside the container.
- 8085 is the **host port**: your browser, cURL, Postman, or Insomnia connect to this port on your machine.
- `-p 8085:5000` means **host port 8085** forwards to **container port 5000**.
- `EXPOSE 5000` documents the internal container port. It does not publish the port by itself.

Port Mapping Rule

```
1 docker run -p HOST_PORT:CONTAINER_PORT image_name
2
3 docker run -p 8085:5000 dsa-lab1-catalog:1.0
```

Remember the difference: before Docker, local Flask testing uses `http://localhost:5000`. After Docker, the containerized service is reached via `http://localhost:8085`.

4 Environment Variables in This Lab

Environment variables are small configuration values available to a process. They are important in distributed systems because the same container image can run in different environments with different settings.

- `PYTHONDONTWRITEBYTECODE=1` tells Python not to create `.pyc` bytecode files.
- `PYTHONUNBUFFERED=1` makes Python write logs directly to the terminal.
- `SERVICE_NAME=catalog-service` gives the application a configurable service name.
- `PORT=5000` defines the port used by Waitress inside the container.

5 Dockerfile and .dockerignore Extras

- `COPY requirements.txt .` before copying the full source helps Docker layer caching.
- `USER appuser` is a better practice than running the application as root.
- `EXPOSE 5000` documents the intended listening port.

.dockerignore

```
1 .git
2 .venv
3 __pycache__/
4 *.pyc
5 .pytest_cache/
6 .env
7 .DS_Store
```

6 HTTP and cURL Mini-Cheat Sheet

- `curl -i <URL>` shows response headers and response body.
- `-X POST`, `-X PUT`, or `-X DELETE` selects an HTTP method explicitly.
- `-H "Content-Type: application/json"` declares that the request body is JSON.
- `-d '{...}'` sends a request body.
- `GET` reads data, `POST` creates data, `PUT` updates data, and `DELETE` removes data.

Local vs. Containerized Test URLs

```

1 # Before Docker: Flask runs directly on your machine
2 curl -i http://localhost:5000/health
3
4 # After Docker: request enters through the published host port
5 curl -i http://localhost:8085/health

```

Further Containerized Test Examples

```

1 # Get one object
2 curl -i http://localhost:8085/items/1
3
4 # Update one object
5 curl -i -X PUT http://localhost:8085/items/1 \
6     -H "Content-Type: application/json" \
7     -d '{"name":"Bella","status":"reserved","species":"dog"}'
8
9 # Delete one object
10 curl -i -X DELETE http://localhost:8085/items/1
11
12 # Trigger an error intentionally
13 curl -i -X POST http://localhost:8085/items \
14     -H "Content-Type: application/json" \
15     -d '{"status":"available"}'

```

7 Useful Git Commands

- `git init` – initialize a new Git repository.
- `git status` – show changed and untracked files.
- `git add <file-or-dir>` – stage changes.
- `git commit -m "message"` – create a commit.
- `git pull` – download remote changes.
- `git push` – upload local commits.

8 Useful Docker Commands

- `docker build --pull -t dsa-lab1-catalog:1.0 .` – build the image from the current directory.
- `docker run -d --name dsa-lab1-catalog -p 8085:5000 dsa-lab1-catalog:1.0` – run the container and publish host port 8085.

- `docker ps` – list running containers.
- `docker logs -f <container-name>` – follow container logs.
- `docker inspect <container-name>` – display detailed metadata.
- `docker exec -it <container-name> /bin/sh` – open a shell inside the running container.
- `docker rm -f <container-name>` – stop and remove a container.

9 Useful Tools

- **cURL**: command-line HTTP client for quick REST tests.
- **HTTPIe**: user-friendly alternative to cURL.
- **Postman or Insomnia**: graphical clients for testing REST APIs.
- **VS Code**: editor with extensions for Python, Docker, and Git.
- **JQ**: command-line JSON formatter and filter.